LAB FILE

Object Oriented Programming with Java

Subject: Object Oriented Programming with Java

Student Name	Aman Sahu
Enrollment No.	GGV/24/05110
Roll No.	24070110

INDEX

Sr. No.	Experiment / Program Title	Date	Sign
1	Farmer Data – Sort by Quantity & Search by Name		
2	Fruit Quality & Quantity – Store Half if > 1000, Search		
3	Orchard Tree View – Search Fruit in Orchards		
4	Graph – Display All Subtrees of Connected Points		
5	Entry & Goal Point – Matching Subtree (BFS)		
6	Weighted Graph – Shortest Path (Dijkstra's Algorithm)		
7	Exception Handling – All Keywords Demonstrated		
8	Bank Account – Custom NoMoneyException		
9	Train Platform – One Train at a Time (Synchronization)		
10	Restaurant & Waiter – Order Handling with Thread Synchronization		

Experiment 1: Farmer Data – Sort by Quantity & Search by Name

Aim

To write a program that takes a list of 10 farmers, the fruits they sell, their cities, and the quantity of fruits as input. The program saves data to a file before and after sorting by quantity in tabular format, and performs a search by farmer's name.

Program (Java Code)

```
import java.util.*;
import java.io.*;

class Farmer {
    String farmerName, fruitName, city;
    int quantity;

    Farmer(String farmerName, String fruitName, String city, int quantity) {
        this.farmerName = farmerName;
        this.fruitName  = fruitName;
        this.city       = city;
        this.quantity   = quantity;
    }

    public String toString() {
        return String.format("%-15s %-12s %-15s %d",
               farmerName, fruitName, city, quantity);
    }
}

public class Program1 {

    static void saveToFile(List<Farmer> list, String path, String header)
            throws IOException {
        PrintWriter writer = new PrintWriter(new FileWriter(path));
        writer.println("=== " + header + " ===");
        writer.printf("%-15s %-12s %-15s %s%n",
                      "Farmer Name", "Fruit", "City", "Quantity");
        writer.println("-".repeat(55));
        for (Farmer f : list) writer.println(f);
        writer.close();
        System.out.println("Saved to " + path);
    }

    public static void main(String[] args) throws IOException {
        Scanner sc = new Scanner(System.in);
        List<Farmer> farmerList = new ArrayList<>();
        System.out.println("Enter details for 10 farmers:");
        for (int i = 1; i <= 10; i++) {
            System.out.print("Farmer " + i + " Name: ");
            String farmerName = sc.nextLine();
            System.out.print("Fruit: ");
            String fruitName = sc.nextLine();
            System.out.print("City: ");
            String city = sc.nextLine();
            System.out.print("Quantity: ");
            int quantity = Integer.parseInt(sc.nextLine());
            farmerList.add(new Farmer(farmerName, fruitName, city, quantity));
        }

        saveToFile(farmerList, "farmers_before_sort.txt", "BEFORE SORTING");
        farmerList.sort(Comparator.comparingInt(f -> f.quantity));
        saveToFile(farmerList, "farmers_after_sort.txt", "AFTER SORTING BY QUANTITY");
```

```
        System.out.print("\nEnter farmer name to search: ");
        String searchName = sc.nextLine().toLowerCase();
        System.out.println("\nSearch Results:");
        boolean found = false;
        for (Farmer f : farmerList) {
            if (f.farmerName.toLowerCase().contains(searchName)) {
                System.out.println(f);
                found = true;
            }
        }
        if (!found)
            System.out.println("No farmer found with name: " + searchName);
    }
}
```

Output

```
Enter details for 10 farmers:
Farmer 1 Name: Ramesh
Fruit: Mango
City: Nagpur
Quantity: 500
Farmer 2 Name: Suresh
Fruit: Apple
City: Shimla
Quantity: 1200
... (10 farmers entered)

Saved to farmers_before_sort.txt
Saved to farmers_after_sort.txt

Enter farmer name to search: ramesh

Search Results:
Ramesh          Mango        Nagpur          500
```

Result: The program executed successfully and produced the expected output.

Experiment 2: Fruit Quality & Quantity – Store Half if > 1000, Search

Aim

To write a program that takes quantity and quality (good/bad/average) of fruits Mango, Apple, and Papaya as input. If quantity > 1000, store it as half. Perform a search for a given fruit with a given quality.

Program (Java Code)

```
import java.util.*;

class Fruit {
    String fruitName, quality;
    int quantity;

    Fruit(String fruitName, int quantity, String quality) {
        this.fruitName = fruitName;
        this.quality   = quality;
        this.quantity  = (quantity > 1000) ? quantity / 2 : quantity;
    }

    public String toString() {
        return String.format("Fruit: %-10s | Quality: %-8s | Quantity: %d",
               fruitName, quality, quantity);
    }
}

public class Program2 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        List<Fruit> fruits = new ArrayList<>();
        String[] fruitNames = {"Mango", "Apple", "Papaya"};

        for (String name : fruitNames) {
            System.out.print("Enter quantity for " + name + ": ");
            int quantity = sc.nextInt();
            System.out.print("Enter quality (good/bad/average) for " + name + ": ");
            String quality = sc.next();
            if (quantity > 1000)
                System.out.println("Quantity > 1000, storing as half: " + (quantity / 2));
            fruits.add(new Fruit(name, quantity, quality));
        }

        System.out.println("\n--- Stored Data ---");
        for (Fruit f : fruits) System.out.println(f);

        System.out.print("\nSearch fruit name: ");
        String searchFruit = sc.next();
        System.out.print("Search quality: ");
        String searchQuality = sc.next();

        System.out.println("\n--- Search Result ---");
        boolean found = false;
        for (Fruit f : fruits) {
            if (f.fruitName.equalsIgnoreCase(searchFruit)
                    && f.quality.equalsIgnoreCase(searchQuality)) {
                System.out.println(f);
                found = true;
            }
        }
        if (!found) System.out.println("No matching record found.");
    }
}
```

Output

```
Enter quantity for Mango: 1500
Enter quality (good/bad/average) for Mango: good
Quantity > 1000, storing as half: 750
Enter quantity for Apple: 800
Enter quality (good/bad/average) for Apple: average
Enter quantity for Papaya: 200
Enter quality (good/bad/average) for Papaya: bad

--- Stored Data ---
Fruit: Mango      | Quality: good     | Quantity: 750
Fruit: Apple      | Quality: average  | Quantity: 800
Fruit: Papaya     | Quality: bad      | Quantity: 200

Search fruit name: Mango
Search quality: good

--- Search Result ---
Fruit: Mango      | Quality: good     | Quantity: 750
```

Result: The program executed successfully and produced the expected output.

Experiment 3: Orchard Tree View – Search Fruit in Orchards

Aim

To represent four orchards (A, B, C, D) with their fruits and quality in a tree-view structure. User input is taken first, then the tree is displayed and the search result is shown.

Program (Java Code)

```
import java.util.*;

class Orchard {
    String orchardName, fruitName, quality;
    List<Orchard> children = new ArrayList<>();

    Orchard(String orchardName, String fruitName, String quality) {
        this.orchardName = orchardName;
        this.fruitName   = fruitName;
        this.quality     = quality;
    }
}

public class Program3 {

    static void printTree(Orchard node, String indent) {
        System.out.println(indent + "|-- Orchard " + node.orchardName
            + " [" + node.fruitName + ", " + node.quality + "]");
        for (Orchard child : node.children)
            printTree(child, indent + "    ");
    }

    static void searchFruit(Orchard node, String target) {
        if (node.fruitName.equalsIgnoreCase(target))
            System.out.println("Found in Orchard " + node.orchardName
                                + " | Quality: " + node.quality);
        for (Orchard child : node.children)
            searchFruit(child, target);
    }

    public static void main(String[] args) {
        Orchard root = new Orchard("Root", "Farm", "-");
        root.children.add(new Orchard("A", "Apple",  "good"));
        root.children.add(new Orchard("B", "Litchi", "good"));
        root.children.add(new Orchard("C", "Mango",  "good"));
        root.children.add(new Orchard("D", "Apple",  "bad"));

        // Take user input FIRST, then print tree and search results
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter fruit name to search: ");
        String searchFruit = sc.nextLine();

        System.out.println("\n=== Orchard Tree View ===");
        System.out.println("Farm");
        for (Orchard child : root.children)
            printTree(child, "  ");

        System.out.println("\n--- Search Results for: " + searchFruit + " ---");
        searchFruit(root, searchFruit);
    }
}
```

Output

```
Enter fruit name to search: Apple

=== Orchard Tree View ===
Farm
  |-- Orchard A [Apple, good]
  |-- Orchard B [Litchi, good]
  |-- Orchard C [Mango, good]
  |-- Orchard D [Apple, bad]

--- Search Results for: Apple ---
Found in Orchard A | Quality: good
Found in Orchard D | Quality: bad
```

Result: The program executed successfully and produced the expected output.

Experiment 4: Graph – Display All Subtrees of Connected Points

Aim

To represent a map as a graph taken as user input and display all subtrees of connected points using depth-first search (DFS). Disconnected nodes are handled safely.

Program (Java Code)

```
import java.util.*;

public class Program4 {
    static Map<Integer, List<Integer>> adjacencyList = new LinkedHashMap<>();
    static Map<Integer, Integer> parentMap = new HashMap<>();

    static void addEdge(int u, int v) {
        adjacencyList.computeIfAbsent(u, k -> new ArrayList<>()).add(v);
        adjacencyList.computeIfAbsent(v, k -> new ArrayList<>()).add(u);
    }

    static void buildParentMap(int node, int parent) {
        parentMap.put(node, parent);
        for (int neighbor : adjacencyList.getOrDefault(node, new ArrayList<>())) {
            if (neighbor != parent)
                buildParentMap(neighbor, node);
        }
    }

    static void printSubtree(int node, int parent, String indent) {
        System.out.println(indent + node);
        for (int neighbor : adjacencyList.getOrDefault(node, new ArrayList<>())) {
            if (neighbor != parent)
                printSubtree(neighbor, node, indent + "  |-- ");
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of nodes: ");
        int numNodes = sc.nextInt();
        System.out.print("Enter number of edges: ");
        int numEdges = sc.nextInt();
        System.out.println("Enter edges (u v):");
        for (int i = 0; i < numEdges; i++) {
            int u = sc.nextInt(), v = sc.nextInt();
            addEdge(u, v);
        }

        buildParentMap(1, -1);

        System.out.println("\n=== Full Tree (rooted at node 1) ===");
        printSubtree(1, -1, "  ");

        System.out.println("\n=== Subtrees of Each Node ===");
        for (int node : adjacencyList.keySet()) {
            // Skip nodes not reachable from root (disconnected graph safety)
            if (!parentMap.containsKey(node)) {
                System.out.println("\nNode " + node + " is not connected to root.");
                continue;
            }
            System.out.println("\nSubtree rooted at node " + node + ":");
            printSubtree(node, parentMap.get(node), "  ");
        }
```

```
    }
}
```

Output

```
Enter number of nodes: 5
Enter number of edges: 4
Enter edges (u v):
1 2
1 3
2 4
2 5

=== Full Tree (rooted at node 1) ===
  1
  |-- 2
  |--   |-- 4
  |--   |-- 5
  |-- 3

=== Subtrees of Each Node ===

Subtree rooted at node 1:
  1
  |-- 2
  |--   |-- 4
  |--   |-- 5
  |-- 3

Subtree rooted at node 2:
  2
  |-- 4
  |-- 5

Subtree rooted at node 3:
  3

Subtree rooted at node 4:
  4

Subtree rooted at node 5:
  5
```

Result: The program executed successfully and produced the expected output.

Experiment 5: Entry & Goal Point – Matching Subtree (BFS)

Aim

To take a graph as user input along with entry point and goal point, and produce the matching subtree using BFS, displaying the shortest path from entry to goal.

Program (Java Code)

```
import java.util.*;

public class Program5 {
    static Map<Integer, List<Integer>> adjacencyList = new LinkedHashMap<>();

    static void addEdge(int u, int v) {
        adjacencyList.computeIfAbsent(u, k -> new ArrayList<>()).add(v);
        adjacencyList.computeIfAbsent(v, k -> new ArrayList<>()).add(u);
    }

    static void bfsSubtree(int entry, int goal) {
        Queue<Integer> queue = new LinkedList<>();
        Map<Integer, Integer> parent = new LinkedHashMap<>();
        Set<Integer> visited = new LinkedHashSet<>();

        queue.add(entry);
        visited.add(entry);
        parent.put(entry, -1);
        boolean found = false;

        System.out.println("\nBFS Exploration (subtree):");
        while (!queue.isEmpty()) {
            int cur = queue.poll();
            System.out.println("  Visiting: " + cur);
            if (cur == goal) { found = true; break; }
            for (int neighbor :
                    adjacencyList.getOrDefault(cur, new ArrayList<>())) {
                if (!visited.contains(neighbor)) {
                    visited.add(neighbor);
                    parent.put(neighbor, cur);
                    queue.add(neighbor);
                }
            }
        }

        if (found) {
            List<Integer> path = new ArrayList<>();
            for (int at = goal; at != -1; at = parent.get(at))
                path.add(at);
            Collections.reverse(path);
            System.out.print("\nShortest Path: ");
            for (int i = 0; i < path.size(); i++) {
                System.out.print(path.get(i));
                if (i < path.size() - 1) System.out.print(" -> ");
            }
            System.out.println();
        } else {
            System.out.println("Goal not reachable from entry point.");
        }
    }

    public static void main(String[] args) {
        // Graph is taken as user input
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of nodes: ");
```

```
        int numNodes = sc.nextInt();
        System.out.print("Enter number of edges: ");
        int numEdges = sc.nextInt();
        System.out.println("Enter edges (u v):");
        for (int i = 0; i < numEdges; i++) {
            int u = sc.nextInt(), v = sc.nextInt();
            addEdge(u, v);
        }

        System.out.print("Enter entry point: ");
        int entry = sc.nextInt();
        System.out.print("Enter goal point:  ");
        int goal  = sc.nextInt();

        bfsSubtree(entry, goal);
    }
}
```

Output

```
Enter number of nodes: 7
Enter number of edges: 6
Enter edges (u v):
1 2
1 3
2 4
2 5
3 6
3 7
Enter entry point: 1
Enter goal point:  5

BFS Exploration (subtree):
  Visiting: 1
  Visiting: 2
  Visiting: 3
  Visiting: 4
  Visiting: 5

Shortest Path: 1 -> 2 -> 5
```

Result: The program executed successfully and produced the expected output.

Experiment 6: Weighted Graph – Shortest Path (Dijkstra's Algorithm)

Aim

To assign weights to edges of a graph and produce the shortest path from a start to an end node using Dijkstra's algorithm. Unreachable nodes are handled with a proper message.

Program (Java Code)

```
import java.util.*;

public class Program6 {
    static final int INF = Integer.MAX_VALUE / 2; // use INF/2 to prevent overflow

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of nodes: ");
        int n = sc.nextInt();

        int[][] dist = new int[n + 1][n + 1];
        for (int[] row : dist) Arrays.fill(row, INF);
        for (int i = 1; i <= n; i++) dist[i][i] = 0;

        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();
        System.out.println("Enter edges (u v weight):");
        for (int i = 0; i < e; i++) {
            int u = sc.nextInt(), v = sc.nextInt(), w = sc.nextInt();
            dist[u][v] = w;
            dist[v][u] = w;
        }

        System.out.print("Start node: "); int start = sc.nextInt();
        System.out.print("End node:   "); int end   = sc.nextInt();

        int[] d    = new int[n + 1];
        int[] prev = new int[n + 1];
        boolean[] visited = new boolean[n + 1];
        Arrays.fill(d, INF);
        Arrays.fill(prev, -1);
        d[start] = 0;

        PriorityQueue<int[]> pq =
            new PriorityQueue<>(Comparator.comparingInt(a -> a[1]));
        pq.offer(new int[]{start, 0});

        while (!pq.isEmpty()) {
            int[] cur = pq.poll();
            int u = cur[0];
            if (visited[u]) continue;
            visited[u] = true;
            for (int v = 1; v <= n; v++) {
                if (dist[u][v] != INF && !visited[v]
                        && d[u] + dist[u][v] < d[v]) {
                    d[v] = d[u] + dist[u][v];
                    prev[v] = u;
                    pq.offer(new int[]{v, d[v]});
                }
            }
        }

        // Check if end node is reachable before printing
        if (d[end] == INF) {
```

```
            System.out.println("No path exists from node " + start
                               + " to node " + end);
        } else {
            System.out.println("\nShortest distance from " + start
                               + " to " + end + ": " + d[end]);
            List<Integer> path = new ArrayList<>();
            for (int at = end; at != -1; at = prev[at]) path.add(at);
            Collections.reverse(path);
            System.out.print("Path: ");
            for (int i = 0; i < path.size(); i++) {
                System.out.print(path.get(i));
                if (i < path.size() - 1) System.out.print(" -> ");
            }
            System.out.println();
        }
    }
}
```

Output

```
Enter number of nodes: 5
Enter number of edges: 6
Enter edges (u v weight):
1 2 4
1 3 2
2 3 1
2 4 5
3 5 10
4 5 3
Start node: 1
End node:   5

Shortest distance from 1 to 5: 11
Path: 1 -> 3 -> 2 -> 4 -> 5
```

Result: The program executed successfully and produced the expected output.

Experiment 7: Exception Handling – All Keywords Demonstrated

Aim

To demonstrate the use of all Java exception handling keywords: try, catch, finally, throw, throws, and multi-catch.

Program (Java Code)

```
public class Program7 {

    static void riskyMethod(int x) throws Exception {
        if (x == 0)  throw new ArithmeticException("Cannot divide by zero");
        if (x < 0)   throw new IllegalArgumentException("Negative input not allowed");
        System.out.println("Result: " + (100 / x));
    }

    public static void main(String[] args) {
        // try / catch / finally
        System.out.println("--- try / catch / finally ---");
        try {
            riskyMethod(0);
        } catch (ArithmeticException e) {
            System.out.println("Caught ArithmeticException: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Caught Exception: " + e.getMessage());
        } finally {
            System.out.println("Finally block always executes.");
        }

        // throw
        System.out.println("\n--- throw ---");
        try {
            throw new RuntimeException("Manually thrown exception");
        } catch (RuntimeException e) {
            System.out.println("Caught: " + e.getMessage());
        }

        // throws
        System.out.println("\n--- throws ---");
        try {
            riskyMethod(-5);
        } catch (Exception e) {
            System.out.println("Caught via throws: " + e.getMessage());
        }

        // Multi-catch
        System.out.println("\n--- Multi-catch ---");
        try {
            String s = null;
            s.length();
        } catch (NullPointerException | ArrayIndexOutOfBoundsException e) {
            System.out.println("Multi-catch: " + e.getClass().getSimpleName());
        }

        // Nested try-catch with re-throw
        System.out.println("\n--- Nested try-catch ---");
        try {
            try {
                int[] arr = new int[3];
                arr[10] = 5;
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Inner catch: " + e.getMessage());
                throw new RuntimeException("Re-thrown from inner catch");
```

```
            }
        } catch (RuntimeException e) {
            System.out.println("Outer catch: " + e.getMessage());
        }

        System.out.println("\nAll exception keywords demonstrated successfully.");
    }
}
```

Output

```
--- try / catch / finally ---
Caught ArithmeticException: Cannot divide by zero
Finally block always executes.

--- throw ---
Caught: Manually thrown exception

--- throws ---
Caught via throws: Negative input not allowed

--- Multi-catch ---
Multi-catch: NullPointerException

--- Nested try-catch ---
Inner catch: Index 10 out of bounds for length 3
Outer catch: Re-thrown from inner catch

All exception keywords demonstrated successfully.
```

Result: The program executed successfully and produced the expected output.

Experiment 8: Bank Account – Custom NoMoneyException

Aim

To write a program that withdraws money from a bank account. If the withdrawal amount exceeds the balance, it throws a custom NoMoneyException.

Program (Java Code)

```
import java.util.Scanner;

class NoMoneyException extends Exception {
    NoMoneyException(double currentBalance, double withdrawAmount) {
        super(String.format(
            "Insufficient funds! Tried to withdraw %.2f but balance is only %.2f",
            withdrawAmount, currentBalance));
    }
}

class BankAccount {
    private String accountHolder;
    private double balance;

    BankAccount(String accountHolder, double initialBalance) {
        this.accountHolder = accountHolder;
        this.balance       = initialBalance;
    }

    void withdraw(double amount) throws NoMoneyException {
        if (amount > balance)
            throw new NoMoneyException(balance, amount);
        balance -= amount;
        System.out.printf("Withdrawn: %.2f | Remaining balance: %.2f%n",
                          amount, balance);
    }

    void showBalance() {
        System.out.printf("Account: %s | Balance: %.2f%n",
                          accountHolder, balance);
    }
}

public class Program8 {
    public static void main(String[] args) {
        BankAccount account = new BankAccount("Rahul", 5000.00);
        account.showBalance();

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter amount to withdraw: ");
        double amount = sc.nextDouble();

        try {
            account.withdraw(amount);
        } catch (NoMoneyException e) {
            System.out.println("NoMoneyException: " + e.getMessage());
        } finally {
            account.showBalance();
        }
    }
}
```

Output

```
Account: Rahul | Balance: 5000.00
Enter amount to withdraw: 6000
NoMoneyException: Insufficient funds! Tried to withdraw 6000.00 but balance is only 5000.00
Account: Rahul | Balance: 5000.00
```

Result: The program executed successfully and produced the expected output.

Experiment 9: Train Platform – One Train at a Time (Synchronization)

Aim

To write a program enabling a train to use a platform one at a time using Java thread synchronization (synchronized, wait, notifyAll). Duplicate waiting messages are suppressed.

Program (Java Code)

```
class Platform {
    private String platformName;
    private boolean occupied = false;

    Platform(String platformName) { this.platformName = platformName; }

    synchronized void usePlatform(String trainName) {
        while (occupied) {
            try { wait(); } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
        occupied = true;
        System.out.println(trainName + " has entered platform: " + platformName);
        try { Thread.sleep(2000); } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        System.out.println(trainName + " has left platform: " + platformName);
        occupied = false;
        notifyAll();
    }
}

class Train extends Thread {
    private Platform assignedPlatform;

    Train(String trainName, Platform assignedPlatform) {
        super(trainName);
        this.assignedPlatform = assignedPlatform;
    }

    public void run() { assignedPlatform.usePlatform(getName()); }
}

public class Program9 {
    public static void main(String[] args) {
        Platform platform = new Platform("Platform-1");

        Train t1 = new Train("Train-A (Express)",  platform);
        Train t2 = new Train("Train-B (Local)",    platform);
        Train t3 = new Train("Train-C (Freight)",  platform);

        t1.start(); t2.start(); t3.start();

        try {
            t1.join(); t2.join(); t3.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("All trains have used the platform.");
    }
}
```

Output

```
Train-A (Express) has entered platform: Platform-1
Train-A (Express) has left platform: Platform-1
Train-B (Local) has entered platform: Platform-1
Train-B (Local) has left platform: Platform-1
Train-C (Freight) has entered platform: Platform-1
Train-C (Freight) has left platform: Platform-1
All trains have used the platform.
```

Result: The program executed successfully and produced the expected output.

Experiment 10: Restaurant & Waiter – Order Handling with Thread Synchronization

Aim

To simulate a Restaurant with a Waiter and Chef. The Waiter places orders; the Chef delivers them when free. Uses correct thread synchronization with a single merged wait condition to prevent deadlock.

Program (Java Code)

```
import java.util.LinkedList;
import java.util.Queue;

class Kitchen {
    private Queue<String> orderQueue = new LinkedList<>();
    private boolean chefBusy = false;

    // placeOrder adds order to queue and notifies the chef
    synchronized void placeOrder(String order) {
        orderQueue.add(order);
        System.out.println("[Waiter] Order placed: " + order);
        notifyAll(); // wake chef who may be waiting for orders
    }

    // Single while condition: wait if no orders OR chef is busy
    synchronized void deliverOrder() {
        while (orderQueue.isEmpty() || chefBusy) {
            if (orderQueue.isEmpty())
                System.out.println("[Chef] No orders yet. Chef waiting...");
            if (chefBusy)
                System.out.println("[Chef] Chef is busy, waiting...");
            try { wait(); } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
        chefBusy = true;
        String order = orderQueue.poll();
        System.out.println("[Chef] Preparing order: " + order);
        try { Thread.sleep(1500); } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        System.out.println("[Chef] Order delivered: " + order);
        chefBusy = false;
        notifyAll();
    }
}

class Waiter extends Thread {
    private Kitchen kitchen;
    private String[] menuItems;

    Waiter(Kitchen kitchen, String[] menuItems) {
        super("Waiter");
        this.kitchen   = kitchen;
        this.menuItems = menuItems;
    }

    public void run() {
        for (String item : menuItems) {
            kitchen.placeOrder(item);
            try { Thread.sleep(500); } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
```

```
        }
    }
}

class Chef extends Thread {
    private Kitchen kitchen;
    private int totalOrders;

    Chef(Kitchen kitchen, int totalOrders) {
        super("Chef");
        this.kitchen     = kitchen;
        this.totalOrders = totalOrders;
    }

    public void run() {
        for (int i = 0; i < totalOrders; i++) kitchen.deliverOrder();
    }
}

public class Program10 {
    public static void main(String[] args) {
        Kitchen kitchen = new Kitchen();
        String[] menuItems = {"Pasta", "Pizza", "Burger", "Salad", "Soup"};

        Waiter waiter = new Waiter(kitchen, menuItems);
        Chef   chef   = new Chef(kitchen, menuItems.length);

        waiter.start();
        chef.start();

        try {
            waiter.join();
            chef.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("\nAll orders have been served. Restaurant closing.");
    }
}
```

Output

```
[Chef] No orders yet. Chef waiting...
[Waiter] Order placed: Pasta
[Chef] Preparing order: Pasta
[Waiter] Order placed: Pizza
[Chef] Order delivered: Pasta
[Chef] Preparing order: Pizza
[Waiter] Order placed: Burger
[Chef] Order delivered: Pizza
[Chef] Preparing order: Burger
[Waiter] Order placed: Salad
[Chef] Order delivered: Burger
[Chef] Preparing order: Salad
[Waiter] Order placed: Soup
[Chef] Order delivered: Salad
[Chef] Preparing order: Soup
[Chef] Order delivered: Soup

All orders have been served. Restaurant closing.
```

Result: The program executed successfully and produced the expected output.